

MODULE 18 · DOMAIN 4 - WORKING WITH LAKEFLOW JOBS

Triggers and scheduling

The five trigger types, cooldown and debouncing, and the choice between time-based and data-driven.

DOMAIN 4 of 7 EXAM WEIGHT 16% MODULE 18 of 36

A time-based trigger runs when the clock says so. A data-driven trigger runs when the data says so. **Almost every scheduling problem is a symptom of using the first where you needed the second.**

The five trigger types

TYPE	FIRES WHEN
Scheduled	A time-based schedule is reached
File arrival	New files arrive in a monitored Unity Catalog storage location
Table update	One or more monitored Unity Catalog tables are updated
Continuous	The previous run completes or fails, keeping the job always running
None (manual)	Someone runs it, or an external orchestrator calls the API

ON THE EXAM

The exam guide names the first three. Note what continuous actually means: a new run starts when the last one ends, whether it succeeded or not.

Scheduled triggers

- ◆ **A time-based schedule**, expressed simply in the UI or as a cron expression for anything unusual.
- ◆ **Set the time zone explicitly**. Otherwise daylight saving will move your job twice a year.
- ◆ **The clock does not know whether the data arrived**. That is the whole limitation.
- ◆ **Still right for genuinely time-bound work**: a report due at 09:00, a month-end close, a regulatory deadline.

ON THE EXAM

A schedule is a promise about when you will start, never a promise about whether there is anything to do.

File arrival triggers

- ◆ **Monitors a Unity Catalog storage location** — an external location or a volume — and fires when new files land.
- ◆ **Enable file events on the external location** for best performance. With file events, there is no limit on the number of files.
- ◆ **The path cannot contain wildcards.** No asterisk, no question mark.
- ◆ **The path must not contain external tables**, or managed locations of catalogs and schemas.

ON THE EXAM

Best practice: create a volume mapped to exactly the subdirectory you want watched, and set the trigger on that volume's root.

Why the subpath advice matters

- ◆ A trigger on a subpath of a large external location still sees changes elsewhere in that location as metadata to process.
- ◆ In a busy environment that can exceed the trigger's processing time limit, putting it into an error state.
- ◆ A dedicated volume scopes the watching precisely, so unrelated activity does not affect it.
- ◆ Configure notifications on job failure to be told when a trigger fails to evaluate.

ON THE EXAM

A trigger that has quietly entered an error state stops firing. Without a notification, nobody finds out until someone asks where the data is.

Table update triggers

- ◆ **Fires when one or more Unity Catalog tables are updated.** Generally available since October 2025.
- ◆ **With several tables, choose the mode:** run when any one updates, or only once all of them have.
- ◆ **Limit of 10 tables per trigger.** A monitored view's source tables count towards that limit.
- ◆ **A separate limit of 10 dependent views** per monitored view.

ON THE EXAM

A view depending on 11 tables cannot be used in a table update trigger at all. Two views of 6 tables each count as 12.

The two timing controls

OPTION	PATTERN	WHAT IT DOES
Minimum time between triggers	Cooldown	At most one run per interval. Arrivals during the cooldown wait.
Wait after last change	Debouncing	Waits this long after the latest change. Each new change resets the timer.

ON THE EXAM

Both options are available on file arrival and table update triggers alike. They solve two different problems.

Using them together

- ◆ Either can be set alone, or both together.
- ◆ With both, the cooldown applies first, then the debounce timer.
- ◆ **Example:** minimum 120 seconds and wait 60 seconds means no run before 120 seconds have elapsed, even if a change happens in the first 60.
- ◆ Debouncing is the answer to 'a batch of 200 files arrived and the job ran 200 times'.

ON THE EXAM

Cooldown protects against running too often. Debouncing protects against running too early, on a partial batch.

Continuous mode

- ◆ A new run starts whenever the previous run completes or fails.
- ◆ Failures are retried automatically with exponential backoff, as module 17 covered.
- ◆ It costs continuously, because something is always running.
- ◆ Serverless jobs support continuous scheduling only with bounded triggers such as `Trigger.AvailableNow`.

ON THE EXAM

Continuous is for genuine streaming. For data arriving in bursts, a file arrival trigger gives similar latency at a fraction of the cost.

Time-based or data-driven?

TIME - BASED

The clock decides

- Predictable, and easy to reason about across teams
- Right when the deadline is the requirement
- Runs whether or not there is anything to do
- Needs a margin, which is waste on good days and too little on bad ones

DATA - DRIVEN

The data decides

- Runs as soon as the data is actually there
- No margin to guess at, and no wasted empty runs
- Lower latency and usually lower cost
- Needs Unity Catalog, and a path or tables to watch

A NAMED OBJECTIVE

The exam guide makes this choice an objective in its own right, phrased as 'based on data availability and pipeline dependencies'.

When time-based is still right

- ◆ **The deadline is the requirement:** a report at 09:00, a month-end close, a regulatory submission.
- ◆ **The source is genuinely periodic** and reliable, and there is nothing to gain from reacting sooner.
- ◆ **You must coordinate with an external system** that expects work in a fixed window.
- ◆ **The source is outside Unity Catalog**, so there is nothing for a data-driven trigger to watch.

ON THE EXAM

Do not treat data-driven as automatically better. Some work genuinely is time-bound, and a schedule expresses that honestly.

When data-driven wins

- ◆ **Arrival times are irregular.** The margin you would need is the problem you are solving.
- ◆ **A downstream job depends on an upstream one** that sometimes runs late.
- ◆ **Empty runs are costing money,** because the schedule fires whether or not anything changed.
- ◆ **Latency matters,** and you want processing to begin the moment data lands.

ON THE EXAM

'Pipeline dependencies' in the objective points at table update triggers. A downstream job watching a table replaces guessing when the upstream finishes.

Concurrency and queueing

- ◆ **Maximum concurrent runs** controls how many runs of the same job can be in flight at once.
- ◆ **The default is one**, so a new trigger while a run is in progress does not start a second run.
- ◆ **Queueing holds the triggered run** rather than dropping it, so it starts when the current one finishes.
- ◆ **Concurrency above one needs idempotent tasks**, or two runs will write over each other.

ON THE EXAM

A job whose runtime exceeds its schedule interval will queue or skip. That is usually the first symptom of a job that has outgrown its window.

Tiebreakers from this module

WHEN A QUESTION SAYS

Files arrive at unpredictable times

A batch of files arrives and the job runs once per file

A source updates constantly and runs are too frequent

A downstream job must wait for an upstream one

More than 10 tables to monitor

A wildcard in the monitored path

A report due at a fixed time

REACH FOR

File arrival trigger

Wait after last change (debouncing)

Minimum time between triggers (cooldown)

Table update trigger on the table it writes

Not possible in one trigger. Split, or watch fewer.

Not supported for file arrival triggers

Scheduled. The deadline is the requirement.

ON THE EXAM

Seven lines. The second and third are the pair most often confused with each other.

A supplier uploads a daily file to a Unity Catalog volume. The delivery time varies between 02:00 and 09:00 depending on their batch. The team currently schedules the ingestion job at 10:00 to be safe.

The business wants the data available sooner. On most days the file has arrived by 03:00, so seven hours are being wasted.

What should the team do?

- ☐ A Move the schedule to 03:00, since that is when the file usually arrives
- ☐ B Use a file arrival trigger on the volume, so the job starts when the file lands
- ☐ C Run the job every 30 minutes from 02:00 until it finds a file
- ☐ D Switch the job to continuous mode

◆ PAUSE AND DECIDE

What should the team do?

A Move the schedule to 03:00
Trades one guess for a worse one. On the days the file arrives at 09:00, the job runs against nothing and the data is missed entirely.

C Poll every 30 minutes
Fourteen runs a day, thirteen of which find nothing, each paying start-up. It reinvents a file arrival trigger badly.

B A file arrival trigger on the volume
Correct. The trigger fires when the file lands, whether that is 02:00 or 09:00. No margin to guess and no wasted wait.

D Continuous mode
Keeps a job permanently running to wait for one file a day. Enormous cost for a daily delivery.

THE SIGNAL

'To be safe' is the phrase to notice. A margin exists because nobody knows when the data arrives, which is exactly what a data-driven trigger removes.

A partner delivers around 200 small files in a burst, over roughly four minutes, twice a day. A file arrival trigger is configured on the volume with no advanced options set.

The job is running close to 200 times per delivery, mostly against partial data, and the compute cost has become the team's largest line item.

What should be configured?

- ☐ A Minimum time between triggers, set to a few minutes
- ☐ B Wait after last change, set to longer than the gap between files in a burst
- ☐ C Maximum concurrent runs, set to 1
- ☐ D Switch to a scheduled trigger twice a day

◆ PAUSE AND DECIDE

What should be configured?

- A

 Minimum time between triggers
A cooldown caps how often runs start, so it helps with cost. It does not wait for the batch to finish, so runs still fire on partial data.
- B

 Wait after last change
Correct. Debouncing waits for quiet, and each new file resets the timer, so one run fires after the whole burst has landed.
- C

 Maximum concurrent runs of 1
Already the default. It prevents overlapping runs, not repeated ones, so 200 runs would simply queue instead.
- D

 A scheduled trigger twice a day
Fixes it by abandoning the data-driven approach and reintroducing the margin problem. It also assumes the deliveries are punctual.

THE SIGNAL

Two symptoms: too many runs, and partial data. A cooldown addresses the first only. Debouncing addresses both.

An ingestion job writes a silver table and usually finishes by 04:20. A downstream aggregation job is scheduled at 05:00 to give it margin. Once or twice a month the ingestion job overruns and the aggregation runs against stale data, producing wrong figures with no error.

Both tables are in Unity Catalog.

What is the best fix?

- ☐ A Move the aggregation job's schedule to 06:00 for more margin
- ☐ B Use a table update trigger on the silver table, so the aggregation starts when the table is written
- ☐ C Combine both jobs into one job with a dependency between the tasks
- ☐ D Add a retry policy to the aggregation job

◆ PAUSE AND DECIDE

What is the best fix?

A More margin

A bigger guess is still a guess. It delays the data every day to protect against a failure that happens twice a month, and eventually overruns again.

C Combine into one job

Genuinely valid, and it couples the two. If the teams own them separately, or the aggregation has other triggers, this is a heavier change.

B A table update trigger on the silver table

Correct. The dependency becomes explicit: the aggregation starts because the table was written, not because the clock reached a time.

D A retry policy

The aggregation is not failing. It succeeds and produces wrong figures, which retries cannot detect.

THE SIGNAL

This is the objective's phrase 'pipeline dependencies' exactly. A dependency encoded as two cron times is a dependency waiting to break.

A team wants a reconciliation job to run only after all fourteen of their regional silver tables have been refreshed. They plan a single table update trigger listing all fourteen, with the mode set to run once all tables have updated.

The trigger cannot be saved.

Why, and what are the options?

- A Table update triggers support only one table; they must use fourteen separate jobs
- B There is a limit of 10 tables per trigger; they must reduce the count, or use a different approach
- C The 'once all tables have updated' mode is not supported; only 'any table' is available
- D Table update triggers require the tables to be in the same schema

◆ PAUSE AND DECIDE

Why, and what are the options?

- A

 Only one table supported
False. Multiple tables are supported, with a choice of any-one or all-of semantics. The constraint is the count, not the concept.
- B

 A limit of 10 tables per trigger
Correct. The documented limit is 10. A monitored view's source tables count towards it too, so views are not a way around it.
- C

 'All tables' mode is unsupported
It is supported, and it is exactly what this team wants. The mode is not the problem.
- D

 Tables must share a schema
Invented restriction. The tables need to be in Unity Catalog, not in one schema.

THE SIGNAL

A view over the fourteen tables does not help: its source tables count towards the same limit of 10.

Ask what the job is actually waiting for. If the answer is **data**, a clock is the wrong instrument. If the answer is **a deadline**, it is the right one.